🏥 HIMS Backend API & Team Integration Handbook

Complete Reference for Frontend, Billing, OPD, Clinical, LIMS & Pharmacy Teams

Testing Server: AWS EC2
Stack: Node.js 20 · Express · Supabase PostgreSQL

> **Testing Environment Notice:** This backend is hosted on a single AWS EC2 instance dedicated for rapid development, testing, and client integration. All teammates can point their frontend interfaces, mobile clients, and Postman collections to the public EC2 endpoint detailed below.

1. AWS EC2 TESTING SERVER SETUP & CLIENT CONNECTION

1.1 How to Host on AWS EC2 with Docker Compose & Caddy (Automatic HTTPS)

The backend is fully containerized with **Docker Compose** and **Caddy Reverse Proxy**, giving you automatic HTTPS/SSL via wildcard DNS (`nip.io`) with zero domain purchases:

Step 1: Configure AWS Security Group (Inbound Rules)
In the AWS EC2 Management Console, open your instance's **Security Group** → **Inbound Rules** and ensure:

- `Port 22 (SSH)` — Source: `My IP` (for remote terminal access)
- `Port 3000 (Custom TCP)` — Source: `0.0.0.0/0` (direct HTTP API access)
- `Port 80 (HTTP)` — Source: `0.0.0.0/0` (Caddy HTTP & Let's Encrypt challenges)
- `Port 443 (HTTPS)` — Source: `0.0.0.0/0` (Caddy Secure HTTPS with SSL)

Step 2: 1-Click Launch on EC2 (via Git Clone)

```
# 1. SSH into your Ubuntu EC2 instance
ssh -i your-key.pem ubuntu@<EC2_PUBLIC_IP>

# 2. Clone the repository
git clone <YOUR_GIT_REPO_URL> hims-backend
cd hims-backend/backend

# 3. Make deploy script executable and run it
chmod +x deploy-ec2.sh
./deploy-ec2.sh

# 4. Update the generated .env with your Supabase credentials:
nano .env

# 5. Re-run deploy to launch the containers:
./deploy-ec2.sh
```

1.2 How Frontend & Teammates Connect to the EC2 Backend

Teammates can connect via either Direct Port 3000 HTTP or Automatic HTTPS using `nip.io` :

```
# Direct HTTP (local testing & Postman):
http://<EC2_PUBLIC_IP>:3000

# Standard HTTP (port 80 via Caddy):
http://<EC2_PUBLIC_IP>

# Automatic HTTPS / SSL (Recommended for Vercel/Netlify Frontends):
https://<EC2_PUBLIC_IP>.nip.io

# Health Check Verification:
https://<EC2_PUBLIC_IP>.nip.io/health
```

Frontend Client Setup (React / Vite / Next.js):
In your frontend `.env.local` file:

```
# If your frontend is deployed on Vercel/Netlify (HTTPS):
VITE_API_URL=https://<EC2_PUBLIC_IP>.nip.io
# If testing locally on localhost:
# VITE_API_URL=http://<EC2_PUBLIC_IP>:3000
```

2. UNIVERSAL API STANDARDS & ERROR CONTRACTS

HTTP STATUS	MEANING	WHEN IT OCCURS
`200 OK`	Success	Standard successful GET or PATCH response.
`201 Created`	Resource Created	Successful POST record creation.
`400 Bad Request`	Validation Error	Invalid payload structure, failed Zod schema checks, missing mandatory fields.
`404 Not Found`	Entity Missing	Requested UUID does not exist in PostgreSQL.
`409 Conflict`	Business Rule Block	Insufficient pharmacy stock during dispense or duplicate unique constraint.
`500 Server Error`	Internal Error	Unhandled exception or database connectivity issue.

Standard Error Envelope

```
{
  "error": "Validation failed: Phone number must match pattern",
  "details": [
    { "field": "phone", "message": "Invalid phone number format" }
  ]
}
```

3. MODULE 1: PATIENT MANAGEMENT API (RECEPTION & REGISTRATION)

Manages patient onboarding, demographic records, and deterministic UHID generation.

POST `/api/patients` AUTO UHID

Register new patient. The database trigger automatically assigns a unique UHID (e.g., `HIMS-2026-00001`).

```
POST /api/patients
Content-Type: application/json

{
  "first_name": "Rohan",
  "last_name": "Sharma",
  "date_of_birth": "1994-08-12",
  "gender": "male",
  "blood_group": "O+",
  "phone": "+919876500001",
  "email": "rohan.sharma@example.com",
  "address": "Flat 302, Green Glen Layout, Bangalore",
  "emergency_contact_name": "Sunita Sharma",
  "emergency_contact_phone": "+919876500002"
}

// Response (201 Created):
{
  "id": "c0000000-0000-0000-0000-000000000001",
  "uhid": "HIMS-2026-00001",
  "first_name": "Rohan",
  "last_name": "Sharma",
  "date_of_birth": "1994-08-12",
  "gender": "male",
  "blood_group": "O+",
  "phone": "+919876500001",
  "is_active": true,
  "created_at": "2026-10-27T10:00:00.000Z"
}
```

GET `/api/patients?search=rohan&page=1&limit=10`

Search patients across first/last names and phone numbers with pagination.

```
// Response (200 OK):
{
  "patients": [ { "id": "...", "uhid": "HIMS-2026-00001", ... } ],
  "pagination": { "page": 1, "limit": 10, "total": 1, "total_pages": 1 }
}
```

GET **/api/patients/:id | /api/patients/uhid/:uhid**

Retrieve a single patient record by internal UUID or hospital UHID string.

PATCH **/api/patients/:id**

Update patient phone, address, or emergency contacts. (UHID is immutable).

DELETE **/api/patients/:id**

Soft delete patient record (sets `is_active: false`).

4. MODULE 2: OPD & APPOINTMENT QUEUE API (OPD & NURSING DESKS)

Handles hospital departments, doctor profiles, appointment bookings, and live queue monitoring.

GET **/api/opd/departments | POST /api/opd/departments**

List active hospital departments or register new department (e.g. `{"name":"Cardiology","code":"CARD"}`).

GET **/api/opd/doctors?departmentId=<UUID>&available=true**

Fetch active doctors filtered by department.

POST **/api/opd/appointments** AUTO DAILY TOKEN

Book an OPD appointment. Database trigger calculates and assigns next sequential token number for that department & date.

```
POST /api/opd/appointments
Content-Type: application/json

{
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "doctor_id": "a0000000-0000-0000-0000-000000000001",
  "department_id": "d0000000-0000-0000-0000-000000000001",
  "scheduled_at": "2026-10-27T10:30:00.000Z",
  "notes": "Follow-up for chest discomfort"
}

// Response (201 Created):
{
  "id": "e0000000-0000-0000-0000-000000000001",
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "department_id": "d0000000-0000-0000-0000-000000000001",
  "token_number": 1,
  "status": "scheduled",
  "scheduled_at": "2026-10-27T10:30:00.000Z"
}
```

PATCH **/api/opd/appointments/:id/status**

Transition appointment status: `scheduled` → `checked_in` → `in_progress` → `completed` (or `cancelled` / `no_show`).

GET **/api/opd/appointments/queue/:departmentId/:date** LIVE QUEUE

Returns the ordered patient queue for a department on a specific date (`YYYY-MM-DD`). Perfect for waiting room monitors.

5. MODULE 3: CLINICAL / EMR API (DOCTOR CONSULTATIONS)

Provides doctors with clinical recording, symptom history, ICD-10 tagging, digital prescriptions, and cross-department referrals.

POST **`/api/clinical`** `FULL CONSULTATION`

Create consultation with embedded prescriptions and referrals in one transaction.

```
POST /api/clinical
Content-Type: application/json

{
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "appointment_id": "e0000000-0000-0000-0000-000000000001",
  "doctor_id": "a0000000-0000-0000-0000-000000000001",
  "chief_complaint": "Mild chest pain and shortness of breath",
  "history_of_illness": "Symptoms started 3 days ago. No prior cardiac history.",
  "examination_notes": "BP: 130/85 mmHg, Pulse: 78 bpm. Heart sounds normal.",
  "diagnosis": "Non-cardiac chest pain, rule out angina",
  "icd10_codes": ["R07.9", "R06.0"],
  "follow_up_date": "2026-11-05",
  "status": "completed",
  "prescriptions": [
    {
      "drug_name": "Aspirin 75mg",
      "dosage": "75mg",
      "frequency": "Once daily after dinner",
      "duration_days": 14,
      "instructions": "Take after meals"
    }
  ],
  "referrals": [
    {
      "referred_to_dept": "Radiology",
      "reason": "Chest X-Ray and ECG",
      "priority": "routine"
    }
  ]
}
```

GET **`/api/clinical/:id`** | **`/api/clinical/appointment/:appointmentId`**

Retrieve complete consultation record with linked prescription items and referrals.

PATCH **`/api/clinical/:id`**

Update diagnosis, follow-up date, or notes.

PATCH **`/api/clinical/prescriptions/:itemId/dispensed`**

Mark an individual prescription item as dispensed (`{"dispensed": true}`).

6. MODULE 4: LABORATORY INFORMATION SYSTEM API (LIMS)

Enables test ordering, specimen tracking, result entry, and secure PDF report access.

GET **`/api/lims/tests`** | **`POST /api/lims/tests`**

Catalog of available lab tests with default units, reference ranges, and test fees.

POST **`/api/lims/orders`**

Place lab order for a patient. Priority can be `routine` , `urgent` , or `stat` .

```
POST /api/lims/orders
Content-Type: application/json

{
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "ordered_by": "a0000000-0000-0000-0000-000000000001",
  "priority": "routine",
  "items": [
    { "test_id": "aa000000-0000-0000-0000-000000000001", "test_name": "Complete Blood Count" }
  ]
}
```

PATCH **/api/lims/orders/:id/status**

Status progression: `pending` → `sample_collected` → `processing` → `resulted` → `delivered`.

PATCH **/api/lims/orders/items/:itemId/result**

Enter numeric or textual test findings: `{"result_value":"14.2","result_unit":"g/dL","reference_range":"13.5-17.5","is_abnormal":false}`.

POST **/api/lims/orders/:id/report**

Attach generated PDF report file path stored in Supabase Storage bucket `lab-reports`.

GET **/api/lims/orders/:id/report/download** SIGNED URL

Generates a secure 5-minute pre-signed download URL from cloud storage. Medical records are never exposed publicly.

7. MODULE 5: PHARMACY & INVENTORY API (DRUG DISPENSING)

Controls medications, batch expiry dates, reorder alerts, and ACID-safe stock deduction.

GET **/api/pharmacy/drugs** | **/api/pharmacy/drugs/alerts/low-stock**

List formulary medications with live stock count or filter drugs strictly below reorder threshold.

POST **/api/pharmacy/stock**

Receive new inventory batch: `{"drug_id":"...","batch_number":"BAT-2026-999","expiry_date":"2027-12-31","quantity":500}`.

POST **/api/pharmacy/dispense** ATOMIC FIFO LOCK

Dispense medications. Calls PostgreSQL `dispense_drugs` PL/pgSQL function. Deducts from batches FIFO by earliest expiry using row locks. Fails with 409 if stock is insufficient.

```
POST /api/pharmacy/dispense
Content-Type: application/json

{
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "dispensed_by": "b0000000-0000-0000-0000-000000000001",
  "items": [
    { "drug_id": "dd000000-0000-0000-0000-000000000001", "drug_name": "Paracetamol 500mg", "quantity": 10 }
  ]
}
```

8. MODULE 6: BILLING & FINANCE API (FOR BILLING TEAM)

Dedicated documentation for managing invoices, multi-department line item charges, and payment tracking.

Billing Workflow Summary:
1. **Create Draft Invoice** (`POST /api/finance`) for a patient visit.
2. **Attach Line Items** (`POST /api/finance/:id/items`) for consultation fees, lab tests, pharmacy dispensing orders.
3. **Issue Invoice** (`PATCH /api/finance/:id/issue`) to finalize total charges.
4. **Record Payments** (`POST /api/finance/:id/payments`). The database automatically updates status to `partially_paid` or `paid` based on real-time balance.

POST `/api/finance` `CREATE INVOICE`

Generate new invoice for patient visit with optional initial charges.

```
POST /api/finance
Content-Type: application/json

{
  "patient_id": "c0000000-0000-0000-0000-000000000001",
  "visit_date": "2026-10-27",
  "discount_amount": 50.00,
  "tax_amount": 25.00,
  "line_items": [
    {
      "charge_type": "consultation",
      "description": "General OPD Consultation",
      "quantity": 1,
      "unit_price": 500.00
    }
  ]
}
```

POST `/api/finance/:id/items` `ADD CHARGES`

Append line items to an open invoice. Automatically recalculates `total_amount`.

```
POST /api/finance/ff000000-0000-0000-0000-000000000001/items
Content-Type: application/json

{
  "items": [
    {
      "charge_type": "lab",
      "description": "Complete Blood Count",
      "quantity": 1,
      "unit_price": 350.00,
      "source_id": "bb000000-0000-0000-0000-000000000001"
    },
    {
      "charge_type": "pharmacy",
      "description": "Paracetamol 500mg (10 tabs)",
      "quantity": 1,
      "unit_price": 40.00
    }
  ]
}
// Allowed charge_type: consultation | lab | pharmacy | procedure | misc
```

POST `/api/finance/:id/payments` `RECORD PAYMENT`

Record payment against invoice. Checks `invoice_summary` view: auto-sets status to `paid` if balance is zero.

```
POST /api/finance/ff000000-0000-0000-0000-000000000001/payments
Content-Type: application/json

{
  "amount": 865.00,
  "method": "upi",
  "reference_no": "UPI-TXN-202610279988",
  "collected_by": "b0000000-0000-0000-0000-000000000001"
}
// Allowed method: cash | card | upi | insurance | waiver
```

GET `/api/finance/:id` FULL INVOICE DETAILS

Retrieves complete invoice record, line items, payments, and real-time SQL balance summary.

GET `/api/finance/summary/patient/:patientId` PATIENT BILL SUMMARY

Direct query against PostgreSQL `invoice_summary` view. Returns subtotal, total, paid, and balance due for every visit.

9. TEST DATA & PRE-SEEDED UUIDS (FOR IMMEDIATE TESTING)

ENTITY	IDENTIFIER / UUID	NAME / DESCRIPTION
Patient 1	`c0000000-0000-0000-0000-000000000001`	Rohan Sharma • UHID: `HIMS-2026-00001` • Phone: `+919876500001`
Patient 2	`c0000000-0000-0000-0000-000000000002`	Priya Patel • UHID: `HIMS-2026-00002` • Phone: `+919876500002`
Doctor (Cardiology)	`a0000000-0000-0000-0000-000000000001`	Dr. Arvind Swaminathan (MD Cardiology)
Doctor (Pediatrics)	`a0000000-0000-0000-0000-000000000002`	Dr. Sunita Rao (MBBS, DCH)
Department (Cardio)	`d0000000-0000-0000-0000-000000000001`	Cardiology (Code: `CARD`)
Lab Test (CBC)	`aa000000-0000-0000-0000-000000000001`	Complete Blood Count (CBC) • Price: ₹350.00
Lab Test (Lipid)	`aa000000-0000-0000-0000-000000000003`	Lipid Profile • Price: ₹750.00
Drug 1	`dd000000-0000-0000-0000-000000000001`	Paracetamol 500mg • Price: ₹4.00/tab
Drug 2	`dd000000-0000-0000-0000-000000000002`	Amoxicillin 500mg • Price: ₹12.50/cap
Staff / Cashier	`b0000000-0000-0000-0000-000000000001`	Admin / Staff collector ID for payments & dispensing

> **Frontend Quickstart Tip:** You can copy and paste the above test UUIDs directly into your frontend code or Thunder Client / Postman collections to immediately test API responses without having to register new entities from scratch!